9. Travelling Salesman Problem (TSP)

Aim

To find a minimum-cost tour that visits every city once and returns to the starting city.

Algorithm

1. Define the cities and their travel-cost matrix.
2. Keep city A as the starting city.
3. Generate every possible order of the remaining cities.
4. Calculate the total cost of each tour, including the return to A.
5. Keep track of the tour with the lowest cost.
6. Print the best path and its minimum cost.

Program

AI.py - C:/Users/Barath/AppData/Local/Programs/Python/Python313/AI.py (3.13.15)

File Edit Format Run Options Window Help

```
from itertools import permutations

cities = ["A", "B", "C", "D"]
cost = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

best_cost = float("inf")
best_path = None

for order in permutations(range(1, len(cities))):
    route = (0,) + order + (0,)
    total = sum(cost[route[i]][route[i+1]]
                for i in range(len(route)-1))
    if total < best_cost:
        best_cost = total
        best_path = route

print("Path:", " -> ".join(cities[i] for i in best_path))
print("Minimum cost:", best_cost)
```

Output

```
>>>
==========
Path: A -> B -> D -> C -> A
Minimum cost: 80
>>>
```

Result

Thus, the minimum-cost tour was found as A → B → D → C → A with total cost 80.